Davante

10

BASE DE DATOS

Operadores y funciones

ÍNDICE

/ 1. Introducción y contextualización práctica 4

/ 2. Operadores. Descripción y conceptos 5

/ 3. Operadores de comparación y aritméticos 5

3.1. Operadores de comparación 5

3.2. Operadores aritméticos 6

/ 4. Operadores de concatenación, lógicos y precedencia 7

4.1. Operadores de concatenación 7

4.2. Operadores lógicos 7

4.3. Precedencia 7

/ 5. Funciones 8

/ 6. Funciones numéricas o matemáticas 9

/ 7. Funciones de caracteres 10

/ 8. Funciones de fechas 11

/ 9. Funciones de conversión, NVL, DECODE y NULLIF 12

9.1. Funciones de conversión 12

9.2. Funciones NVL, DECODE y NULLIF 12

© MEDAC ISBN: 978-84-19885-68-5
Reservados todos los derechos. Queda rigurosamente prohibida, sin la autorización escrita de los titulares del copyright, bajo las sanciones establecidas en las leyes, la reproducción, transmisión y distribución total o parcial de esta obra por cualquier medio o procedimiento, incluidos la reprografía y el tratamiento informático.

ÍNDICE

/ 10. La expresión CASE 13

/ 11. Funciones analíticas y de ventana 14

11.1. La cláusula OVER 14

11.2. Funciones de ventana 14

/ 12. Caso práctico 1: "Uso de operadores en Oracle Express" 15

/ 13. Caso práctico 2: "Uso de funciones en Oracle Express/Oracle SQL Live. Funciones" 16

/ 14. Resumen y resolución del caso práctico de la unidad 16

/ 15. Bibliografía 18

OBJETIVOS

Familiarizarse con los conceptos de función y operador en una BD.

Aprender a utilizar y entender el uso de los distintos tipos de operadores como complemento a los comandos y las sentencias de una BD.

Comprender el uso de diversas funciones para efectuar operaciones en los datos de una BD.

Ejecutar consultas más complejas a través del uso de funciones y operadores.

/ 1. Introducción y contextualización práctica

A lo largo de la asignatura, hemos explorado la amplia variedad de operaciones posibles en una BD. Para simplificar la creación de consultas más complejas, se emplean funciones que dependen del SGBD utilizado, ya que están predefinidas por este. Centraremos nuestro estudio en las funciones predefinidas en Oracle, estudiando las principales y más usadas.

Fig.1. Los operadores de concatenación son uno de los diversos tipos de operadores SQL.

Para aprovechar al máximo las capacidades de un SGBD y ejecutar tanto consultas simples como avanzadas, es esencial dominar el uso de operadores y funciones. Estos permiten efectuar operaciones de manera fácil y cómoda, facilitando la búsqueda de información en una BD.

Respecto a los **operadores**, hay varios tipos según la operación que se desee realizar, como concatenación, comparación, etc. Se usan comúnmente en consultas como SELECT, además de ser habituales en otros contextos, como en distintos lenguajes de programación.

Las **funciones**, por su parte, se suelen utilizar para realizar cálculos matemáticos, convertir entre diferentes tipos de datos o manejar fechas en una BD. Estas tareas son habituales en el uso diario de este tipo de sistemas.

A continuación, vamos a plantear un caso práctico a través del cual podremos aproximarnos de forma práctica a la teoría de este tema.

Escucha el siguiente audio donde planteamos la contextualización práctica de este tema. Encontrarás su resolución en el apartado 'Resumen y resolución del caso práctico'.

Audio intro. "Manejo de operadores en una base de datos"

https://on.soundcloud.com/z8VztsNcv1Qhe2iw9

/ 2. Operadores. Descripción y conceptos

Cuando se consulta información en una base de datos, es frecuente que sea necesario establecer comparaciones sobre los datos. Como se mencionó en el tema anterior, esto se lleva a cabo utilizando la cláusula WHERE. Además, a menudo es necesario emplear relaciones y realizar comparaciones lógicas. Para estos propósitos, se deben usar operadores.

Se pueden diferenciar los siguientes tipos de operadores (imagen):

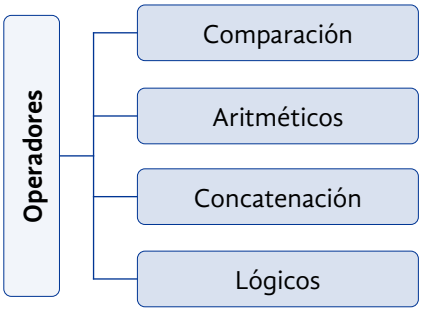

Fig.2. Tipos de operadores.

Algunos de estos operadores ya los hemos visto en ejemplos que nos permitieron explicar las cláusulas vistas en el tema de 'La sentencia SELECT', como, por ejemplo, el de la siguiente imagen:

```
SELECT OrderID,SUM(Quantity)
FROM OrderDetails
WHERE OrderID = 10248 OR OrderID = 10249
GROUP BY OrderID;
```

Fig.3. Ejemplo de sentencia SELECT.

En este ejemplo, podemos ver el uso de distintos tipos de operadores para obtener los datos necesarios. Específicamente, se están combinando dos operadores de comparación (=) con un operador lógico (AND). Esta combinación es bastante común en las consultas a bases de datos.

Una característica importante de los operadores es que pueden combinarse y utilizarse según las necesidades del usuario. En este tema, se analizarán en profundidad todos los tipos de operadores disponibles.

Los operadores son elementos esenciales en las sentencias SQL. Se emplean en casi todas las consultas SELECT que hacen uso de la cláusula WHERE, además de en otros tipos de consultas que se abordarán en este y en posteriores temas. Es importante resaltar que su uso facilita que las sentencias sean más comprensibles y fáciles de implementar.

/ 3. Operadores de comparación y aritméticos

Estos tipos de operadores son los más básicos que podemos utilizar.

3.1. Operadores de comparación

Tal y como indica su nombre, los operadores de comparación permiten **realizar comparaciones entre datos**, sean del tipo que sean. Algunos de estos operadores se representan con símbolos comunes y otros con palabras específicas de SQL.

A continuación podrás ver un recurso sobre el punto anterior:

Recuerda...

Este tipo de operadores lo hemos estudiado en el tema '6. Elaboración de diagramas de entidad-relación. Conceptos del modelo relacional' en su sección de 'Operadores lógicos, operadores de comparación y criterios de ordenación'.

En las siguientes imágenes, se muestran varios ejemplos de **uso de operadores de comparación** como LIKE, > e IN:

```
SELECT NOMBRE FROM EMPLEADOS
WHERE NOMBRE LIKE 'J%';
```

Fig.4. Ejemplos de uso de operadores de comparación 1.

```
SELECT IDPRODUCTO FROM ALMACEN
WHERE PRECIO > 100;
```

Fig.5. Ejemplos de uso de operadores de comparación 2.

```
SELECT NOMBRE, APELLIDO
FROM EMPLEADOS
WHERE NOMBRE IN ('JAVIER', 'ANA', 'JAIME');
```

Fig.6. Ejemplos de uso de operadores de comparación 3.

3.2. Operadores aritméticos

Los operadores aritméticos, familiares para todos, se presentan en la siguiente tabla:

Operador	TEXTO
+	Suma
-	Resta
*	Multiplicar
/	Dividir

Tabla 1. Operadores aritméticos.

Cuando se aplican a campos con valor NULL, el resultado también será NULL. Estos operadores **permiten ejecutar consultas que devuelvan el resultado directamente**, como se ilustra en la siguiente imagen:

```
SELECT PRECIO, PRECIO*1,10
FROM ALMACEN
WHERE PRECIO > 100;
```

Fig.7. Ejemplo de sentencia con cálculo incluido.

Esto se denomina 'campo calculado' y tiene la característica de no modificar los valores del campo original. Por ejemplo, la columna PRECIO de la imagen anterior no será alterada. En su lugar, se crea un nuevo campo con un nombre basado en la expresión utilizada que puede ser renombrado usando el comando AS.

/ 4. Operadores de concatenación, lógicos y precedencia

Los operadores de concatenación, lógicos y la precedencia de operadores juegan un papel crucial en la manipulación y consulta de datos.

4.1. Operadores de concatenación

El operador de concatenación en SQL es '||'. Un ejemplo de su uso es la concatenación del nombre y el fabricante de un producto del almacén en una misma columna, como se muestra en la siguiente imagen:

```
SELECT IDPRODUCTO, NOMBRE || FABRICANTE
FROM ALMACEN
WHERE PRECIO > 100;
```

Fig.8. Ejemplo de sentencia utilizando operador de concatenación.

4.2. Operadores lógicos

Los operadores lógicos **permiten evaluar múltiples expresiones dentro de una sola sentencia**. Estos operadores verifican si varias condiciones se cumplen simultáneamente o si alguna de ellas se cumple dentro de un grupo.

En la siguiente tabla se muestran estos operadores:

Operador	Operación
AND	TRUE si se cumplen todas las condiciones.
OR	TRUE si se cumple alguna de las condiciones.
NOT	Si la expresión es TRUE, devuelve FALSE, y viceversa.

Tabla 2. Operadores lógicos.

En la siguiente imagen, se presenta un ejemplo de uso de operadores lógicos, aunque ya se han visto otros ejemplos a lo largo del curso.

```
SELECT ID_EMPLEADO, NOMBRE, APELLIDOS
FROM EMPLEADOS
WHERE SUELDO>1000 AND SUELDO<500
```

Fig.9. Ejemplo del uso de operadores lógicos.

4.3. Precedencia

La precedencia **determina el orden en el que se evalúan las partes de una sentencia**, por lo que es esencial tenerla en cuenta. En Oracle, el orden de precedencia es el siguiente:

- Operadores '*' y '/'.
- Operadores '+' y '–'.
- Operador de **concatenación** '||'.
- Operadores de **comparación** ('<', '>', '=', etc.).
- Operadores '**IS NULL**', '**NOT NULL**', '**BETWEEN**'.
- '**NOT**'.
- '**AND**'.
- '**OR**'.

Sin embargo, este orden puede ser modificado mediante el uso de paréntesis, como sucede en una operación matemática.

Vídeo 1. "Ejemplos de precedencia de operadores"

https://bit.ly/30S3bPO

/ 5. Funciones

Las funciones son **bloques de código en SQL que realizan operaciones específicas y devuelven un resultado**. Permiten realizar operaciones sobre un conjunto de parámetros de entrada especificados entre paréntesis. Algunas funciones, como SYSDATE, no necesitan parámetros.

En la siguiente imagen, podemos ver la sintaxis que siguen las funciones:

nombreFunción [parámetro1 [, parámetro2, ...])]

Fig.10. Sintaxis de uso de una función.

Se utilizan tanto con el comando SELECT como con las cláusulas WHERE, ORDER BY, etc., y pueden anidarse para que el parámetro de entrada de una función sea el resultado de esta última.

Aunque SQL define una serie de funciones para el tratamiento de datos, cada SGBD puede definir sus propias funciones o incluso personalizar las del estándar SQL. En nuestro caso, nos centraremos en las funciones de Oracle, que incluyen tipos numéricos, de fechas y de conversión.

Audio 1. "Tipos de funciones Oracle"

https://bit.ly/2X8GBRP

Para poder familiarizarse con las funciones, es importante conocer la existencia de una tabla que Oracle denomina DUAL. Esta tabla, con una sola columna (DUMMY) y una fila, se utiliza para realizar pruebas antes de ejecutar consultas sobre datos reales, previniendo de pérdidas de información y otros problemas.

Podemos usar la tabla DUAL para probar todas las funciones que se estudiarán en este tema, así como otros cálculos. Para referirse a ella, basta con incluir DUAL en la cláusula FROM de la sentencia, como se muestra en el ejemplo de la siguiente imagen:

```
select * from DUAL;
```

Fig.11. Ejemplo de uso de la tabla DUAL de Oracle.

En la siguiente imagen, podemos ver el resultado de esta consulta:

DUMMY
X

Fig.12. Resultado de la consulta anterior.

/ 6. Funciones numéricas o matemáticas

Las funciones numéricas se utilizan para **obtener el resultado de realizar determinados cálculos sobre un número**.

Función	Descripción
MOD (num1, num2)	Calcula el resto de dividir el parámetro num1 entre el parámetro num2.
POWER (num1, exp)	Eleva num1 al exponente exp.
SQRT (num)	Calcula la raíz cuadrada del número pasado por parámetro.
SIGN (num)	En función del signo de num, devuelve 1 si es positivo, 0 si es 0 y -1 si es negativo.
ABS (num)	Devuelve el valor absoluto de num.
EXP (num)	Se utiliza para calcular el exponente en base e del número num.
LN (num)	Calcula el logaritmo neperiano del número num.
LOG (num)	Calcula el logaritmo en base 10 del número num.

Tabla 3. Funciones numéricas o matemáticas.

Función	Descripción
MOD (num1, num2)	Devuelve el resto de dividir num1 entre num2.
ROUND (num, nº decimales)	Redondea un número decimal con el número de decimales indicado.
TRUNC (num, nº decimales)	Retorna el número decimal num con el número de decimales indicado, descartando el resto de decimales que tenga.
CEIL (num)	Redondea el valor al entero inmediatamente superior o igual a num.
FLOOR (num)	Igual a CEIL, pero redondeando al entero inmediatamente inferior.

Tabla 4. Funciones para tratamiento de decimales.

Función	Descripción
SIN (num)	Permite calcular el seno del ángulo num, expresado en radianes.
COS (num)	Permite calcular el coseno del ángulo num, expresado en radianes.
TAN (num)	Permite calcular la tangente del ángulo num, expresado en radianes.
ACOS (num)	Calcula en radianes el arco coseno del ángulo num expresado en radianes.
ASIN (num)	Calcula en radianes el arco seno del ángulo num expresado en radianes.

Tabla 5. Funciones trigonométricas.

Investigamos...

Aunque hemos visto ya una gran cantidad de funciones para trabajar con datos numéricos y realizar operaciones matemáticas, existen todavía más. Investiga utilizando Google Scholar para profundizar más tus conocimientos.

/ 7. Funciones de caracteres

Las funciones de caracteres se usan para **realizar operaciones sobre cadenas de caracteres**, en vez de sobre números.

Función	Descripción
LOWER (abc)	Convierte a minúsculas el texto proporcionado como parámetro.
UPPER (abc)	Igual que el anterior, pero lo pasa a mayúsculas.
INITCAP (abc)	Convierte a mayúsculas la primera letra de cada palabra.
RTRIM (abc)	Elimina los espacios a la derecha del texto.
LTRIM (abc)	Elimina los espacios a la izquierda del texto.

Tabla 6. Funciones de caracteres (1).

Función	Descripción
TRIM (abc)	Elimina los espacios a derecha e izquierda.
TRIM (abc) FROM (columna)	Elimina los caracteres indicados en TRIM de una columna determinada.
SUBSTR (abc, num1, num2)	Devuelve los num2 caracteres siguientes del texto, empezando en el carácter de la posición num1 (incluido en el resultado). Ejemplo: "SELECT SUBSTR('Luna',1,2) FROM DUAL" devolvería el texto "Lu".
LENGTH (abc)	Devuelve la cantidad de caracteres de la cadena proporcionada como parámetro.
REPLACE (abc, Texto1, Texto2)	Busca en abc el texto especificado en Texto1 y lo reemplaza por el indicado en Texto 2. Ejemplo: "SELECT REPLACE(`RISA','SA','MA') FROM DUAL" devuelve el resultado "RIMA".

Tabla 7. Funciones de caracteres (2).

Función	Descripción
REVERSE (abc)	Devuelve en orden inverso el texto pasado como parámetro.
INSTR (abc, texto1)	Devuelve la posición en la que se encuentra el texto1 dentro del texto abc.
LPAD (texto1, long, texto2)	Devuelve el texto texto1 con una longitud long, rellenando por la izquierda con texto2. Ejemplo: "SELECT LPAD('1','5','D') FROM DUAL" devolvería como resultado "DDDD1".
RPAD (texto1, long, texto2)	Devuelve el texto texto1 con longitud long, rellenando por la derecha con texto2. Ejemplo: "SELECT RPAD('1','5','D') FROM DUAL" devolvería como resultado "1DDDD".

Tabla 8. Funciones de caracteres (3).

/ 8. Funciones de fechas

El manejo de fechas es una práctica muy común y útil a la hora de trabajar con una base de datos. En Oracle se emplean dos tipos de datos diferentes para manejar las fechas:

- **DATE**: Permite almacenar fechas específicas.
- **TIMESTAMP**: Guarda momentos exactos, incluyendo la fecha y la hora con una precisión de hasta fracciones de segundo.

Se pueden realizar operaciones de suma o resta con las fechas. En estas operaciones **si el parámetro es un número, este se interpreta como el número de días que se quiere añadir o restar**. De igual modo, **cuando se suman o restan fechas entre sí, el resultado se expresa en número de días**.

Por ejemplo, la consulta "SELECT SYSDATE - 3;" devolverá la fecha que fue 3 días antes de la actual.

En las siguientes tablas se pueden ver las funciones más habituales para el manejo de fechas en Oracle.

Función	Descripción
SYSDATE	Muestra la fecha y hora actual.
SYSTIMESTAMP	Muestra la fecha y hora actuales en formato TIMESTAMP.

Tabla 9. Funciones para obtención de fechas.

Función	Descripción
ADD_MONTHS (fecha, num)	Devuelve la fecha resultante de agregar un número específico de meses a una fecha dada. Ejemplo: "SELECT ADD_MONTHS('09/05/2015', 3) FROM DUAL;" devuelve 09/08/2015.
MONTHS_BETWEEN (Fecha1, Fecha2)	Calcula el número de meses entre los parámetros Fecha1 y Fecha2.
LAST_DAY (fecha)	Devuelve el último día del mes de una fecha dada en formato DATE. Ejemplo: "SELECT LAST_DAY(09/05/2015) FROM DUAL;" devuelve 09/05/2015.
NEXT_DAY (fecha, día)	Devuelve el siguiente día específico (lunes, martes, etc.) a partir de una fecha dada. Puede usarse el nombre del día o su número (1 = lunes, 2 = martes, etc.). Ejemplo: "SELECT NEXT_DAY('09/05/2024', 'LUNES') FROM DUAL;" devuelve 09/05/2024, el lunes siguiente al 12/05/2024.
EXTRACT (valor FROM fecha)	Extrae un componente específico (día, mes, año, hora) de una fecha. Ejemplo: "SELECT EXTRACT(month FROM '09/05/2015');" devuelve 5.

Tabla 10. Funciones para cálculos con fechas.

/ 9. Funciones de conversión, NVL, DECODE y NULLIF

A la hora de realizar el tratamiento de datos, es habitual necesitar una transformación sobre los mismos.

9.1. Funciones de conversión

Las funciones de conversión **permiten convertir un tipo de dato en otro**, como convertir un número en texto. En la siguiente tabla, se muestran algunas de las funciones de conversión más utilizadas:

Función	Descripción
TO_CHAR (expr, [formato])	Convierte una fecha o número en texto o carácter. Se puede incluir un parámetro opcional para especificar el formato de conversión (la manera de realizarla). Ejemplo: "SELECT TO_CHAR(SYSDATE, 'DD/MONTH/YYYY') FROM DUAL;"
TO_DATE (expr, [formato])	Convierte un texto en una fecha.
TO_NUMBER (expr, [formato])	Convierte un texto en un número.
CAST (exp AS tipo)	Convierte una expresión en un tipo de datos específico.

Tabla 11. Funciones de conversión.

Vídeo 2. "Ejemplo de uso de funciones"
https://bit.ly/2X7mjbE

9.2. Funciones NVL, DECODE y NULLIF

Las funciones NVL y DECODE **se utilizan para gestionar celdas con valores NULL**. En la siguiente tabla, se presentan estas funciones para el manejo de nulos:

Función	Descripción
NVL (valor1, valor2)	Si valor1 es NULL, devuelve valor2. De lo contrario, devuelve valor1. Ejemplo: "SELECT NVL (NOMBRE, 'Sin nombre') FROM CLIENTES;" devuelve el nombre de todos los clientes, y si no lo tienen (valor NULL), devuelve el literal 'Sin nombre'.
NVL2 (valor1, valor2, valor3)	Si valor1 no es NULL, devuelve valor2. Si valor1 es NULL, devuelve valor3.
NULLIF (valor1, valor2)	Devuelve NULL si valor1 y valor2 son iguales. De lo contrario, devuelve valor1.
DECODE (expr, cond1, result1 [cond2, result2, ...], default)	Evalúa la expresión expr o campo. Si cumple con cond1, devuelve result1. Si no, evalúa la siguiente condición, y así sucesivamente. Si expr no cumple con ninguna de las cond, devuelve default. Ejemplo: "SELECT DECODE(5+3, 10, 'Igual a 10', 8, 'Menor que 10', 'Otro valor') FROM DUAL;" evalúa la expresión aritmética 5+3, como da 8, devuelve 'Menor que 10'.

Tabla 12. Funciones NVL y DECODE.

¡! Sabías que...

La función DECODE cumple un fin similar al de la estructura de control de flujo condicional 'IF-THEN-ELSE' que podemos encontrar en muchos lenguajes de programación.

/ 10. La expresión CASE

La expresión **CASE es una expresión que tiene un comportamiento parecido a la función DECODE** que hemos visto anteriormente. Nos **permite evaluar un conjunto de condiciones y mostrar un resultado dependiendo de cuál de las condiciones se cumpla** de una forma más potente y flexible que DECODE. Debido a esto, **CASE es ideal para implementar lógica condicional avanzada en consultas SQL de manera clara y eficiente**.

La expresión **CASE forma parte del estándar SQL**, su implementación y funcionalidad son muy similares en diferentes sistemas de bases de datos, incluyendo Oracle SQL, DB2, SQL Server, PostgreSQL y MySQL, entre otros. Sin embargo, siempre puede haber pequeñas variaciones o extensiones específicas en cada SGBD.

En la siguiente imagen se muestra su sintaxis:

```
CASE
  WHEN condición1 THEN resultado1
  WHEN condicion2 THEN resultado2
  ...
  ELSE resultado_por_defecto
END
```

Fig.13. Sintaxis de CASE.

En la siguiente imagen se puede ver un ejemplo de su uso real:

```
SELECT titulo,
CASE
 WHEN autor IS NULL THEN 'Anónimo'
 ELSE autor
 END AS autor
FROM libros;
```

Fig.14. Ejemplo de uso de CASE.

En la siguiente imagen se puede ver un ejemplo del resultado obtenido ejecutando la consulta anterior:

	TITULO	AUTOR
1	El señor de los anillos	J.R.R. Tolkien
2	El hobbit	J.R.R. Tolkien
3	Cien años de soledad	Gabriel García Márquez
4	El amor en los tiempos del cólera	Gabriel García Márquez
5	Lazarillo de Tormes	Anónimo

Fig.15. Ejemplo de resultado con autor anónimo en el último libro, ya que no tiene autor

/ 11. Funciones analíticas y de ventana

Las funciones analíticas, también conocidas como funciones de ventana, **realizan cálculos basados en un grupo de filas (ventana) y devuelven un valor para cada fila en el grupo**. Se utilizan con la cláusula OVER que define la ventana de filas a la que se aplica la función.

Investigamos...

Existen una gran cantidad de funciones analíticas, de ventana y OLAP. Te recomendamos que investigues qué otras funciones existen y que pruebes a ejecutar algunas de ellas en tu base de datos para probar su comportamiento y resultados.

11.1. La cláusula OVER

La cláusula OVER se utiliza para definir la ventana de filas a las que se aplica la función analítica. Puede incluir las siguientes partes:

- **PARTITION BY**: Divide las filas en particiones para el cálculo.
- **ORDER BY**: Define el orden de las filas dentro de cada partición.
- **ROWS/RANGE**: Define un rango de filas alrededor de la fila actual.

11.2. Funciones de ventana

Dentro de este tipo de funciones podemos destacar las siguientes:

Función	Descripción
ROW_NUMBER	Asigna un número de fila único a cada fila dentro de la partición de un conjunto de resultados.
RANK	Asigna un rango a cada fila dentro de la partición de un conjunto de resultados, con saltos en los rangos para filas con valores iguales.
DENSE_RANK	Similar a RANK (), pero sin saltos en los rangos.
NTILE(n)	Divide el conjunto de resultados en n grupos aproximadamente iguales y asigna un número de grupo a cada fila.

Tabla 13. Algunas funciones de ventana.

En las siguientes imágenes, podemos ver un ejemplo de la función ROW_NUMBER () aplicada sobre una tabla de libros y su resultado.

```
SELECT
    autor,
    titulo,
    año_publicacion,
    ROW_NUMBER() OVER (
        PARTITION BY autor
        ORDER BY año_publicacion asc) AS RN
FROM libros;
```

Fig.16. Ejemplo de consulta con ROW_NUMBER.

A continuación podrás ver un recurso sobre el punto anterior:

	AUTOR	TITULO	AÑO	RN
1	Gabriel García Márquez	Cien años de soledad	1967	1
2	Gabriel García Márquez	El amor en los tiempos del cólera	1985	2
3	J.R.R. Tolkien	El hobbit	1937	1
4	J.R.R. Tolkien	El señor de los anillos	1954	2
5	(null)	Lazarillo de Tormes	1554	1

Fig.17. Resultado de ejecución.

/ 12. Caso práctico 1: "Uso de operadores en Oracle Express"

Planteamiento: Vamos a poner en práctica los operadores estudiados en este tema, aplicándolos a nuestra base de datos de la tienda de informática dentro de un entorno real. Esta aplicación nos permitirá observar cómo funcionan y se comportan estos operadores con datos concretos.

Nudo: Para este propósito, emplearemos las tablas que hemos creado en temas anteriores, utilizando ya sea la plataforma Oracle Express u Oracle SQL Live. Procederemos a ejecutar diversas consultas sobre estas tablas, similares a las vistas en el tema, en las que aplicaremos los operadores estudiados para extraer información relacionada con clientes, almacén y empleados. Es necesario realizar al menos una consulta de cada tipo, procurando, además, combinar varios operadores en una misma consulta para obtener resultados más complejos y útiles.

Desenlace: En este caso, el objetivo es llevar a cabo consultas similares a las presentadas en los vídeos, aplicándolas a la base de datos de la tienda de informática que hemos construido a lo largo del curso.

En la siguiente imagen, se presenta un ejemplo de una consulta que utiliza operadores sobre la tabla Empleados.

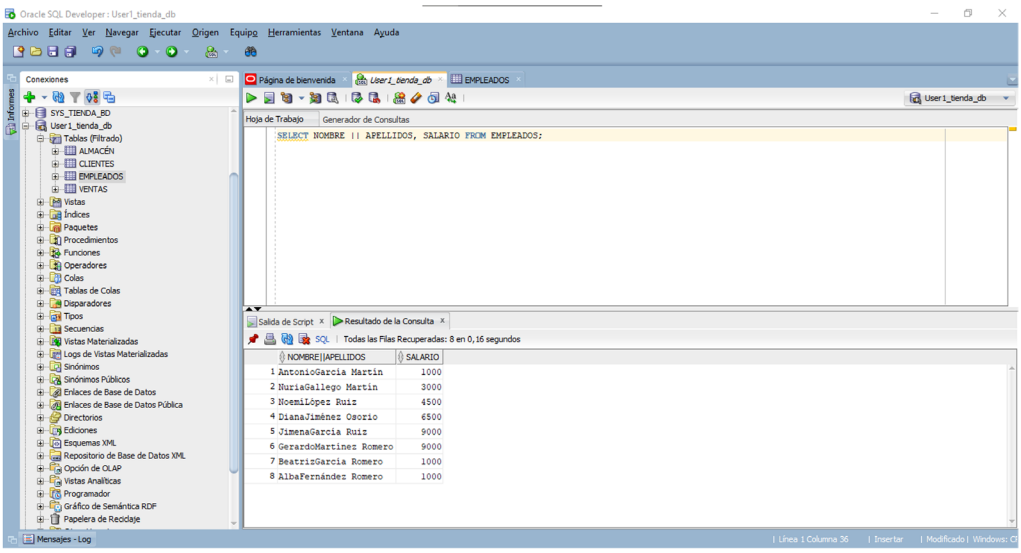

Fig.18. Ejemplo de uso de operadores en SQL Developer.

/ 13. Caso práctico 2: "Uso de funciones en Oracle Express/Oracle SQL Live. Funciones"

Planteamiento: Ahora probaremos el uso de las funciones estudiadas en este tema en un entorno real de base de datos. Al igual que en el caso práctico 1, trabajaremos con las tablas de la base de datos de la tienda de informática que hemos ido creando, pero esta vez nos centraremos en aplicar las funciones aprendidas.

Nudo: Para poner en práctica el conocimiento adquirido sobre funciones, emplearemos las tablas creadas en temas anteriores, utilizando ya sea la plataforma Oracle Express u Oracle SQL Live. Realizaremos diversas consultas sobre estas tablas, aplicando las funciones estudiadas previamente. Debemos hacer al menos una consulta de cada tipo que sirva como ejemplo de su uso.

Desenlace: En esta ocasión, el objetivo también es ejecutar consultas que permitan practicar los ejemplos presentados en los vídeos, aplicadas a la base de datos de la tienda de informática que hemos construido. Con ello conseguiremos consolidar nuestros conocimientos al ver cómo se comportan con datos específicos.

En la siguiente imagen, se muestra una consulta realizada sobre la tabla Empleados, utilizando funciones.

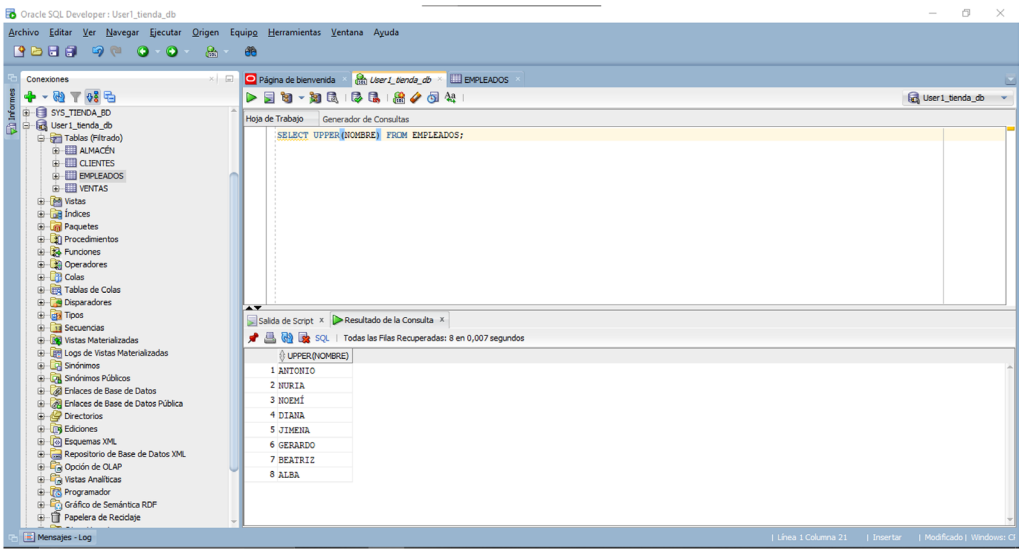

Fig.19. Ejemplo de uso de funciones en SQL Developer.

/ 14. Resumen y resolución del caso práctico de la unidad

En este tema, hemos aprendido a usar operadores y funciones para optimizar bases de datos. Además, hemos visto operadores de comparación, aritméticos, de concatenación y lógicos, así como su precedencia.

También hemos estudiado funciones con la tabla DUAL de Oracle y abordado funciones matemáticas, de caracteres, de fecha, de conversión, NVL, DECODE, CASE, y funciones analíticas y de ventana para cálculos avanzados.

A continuación podrás ver el esquema resumen del tema:

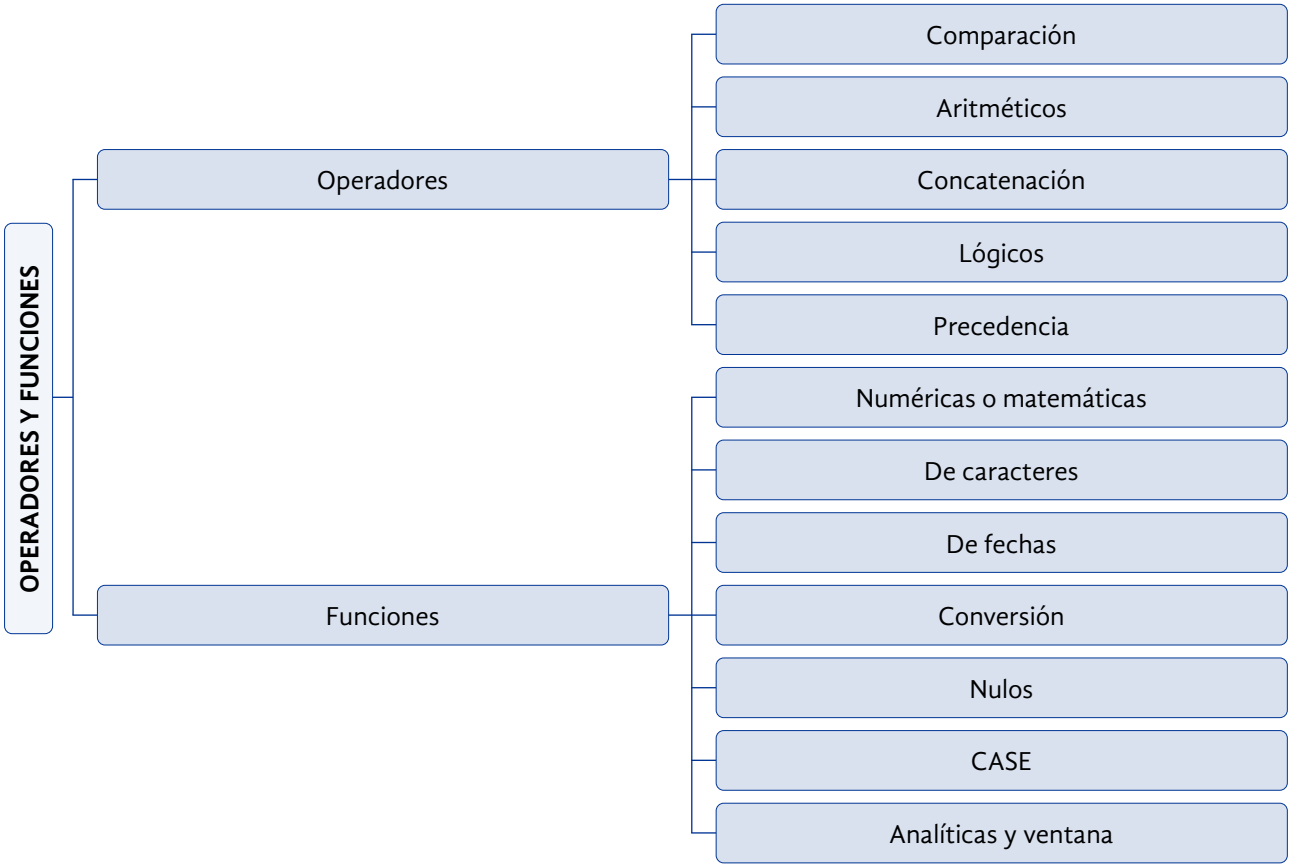

Fig.20. Esquema resumen del tema .

Resolución del caso práctico de la unidad

En el caso práctico planteado en el audio de introducción al tema, se exponía la necesidad de saber algunos datos relacionados con la tienda de informática. A continuación, se indican las sentencias SQL necesarias para extraer esa información:

- ¿Qué clientes se llaman Antonio?

  SELECT * FROM CLIENTES WHERE NOMBRE = 'ANTONIO';

- ¿Cuántos de sus empleados tienen un nombre que empieza por H?

  SELECT count(*) FROM EMPLEADOS WHERE NOMBRE LIKE 'H%';

- ¿Cuáles de ellos tienen un salario superior a 1.000?

  SELECT * FROM EMPLEADOS WHERE NOMBRE LIKE 'H%' AND SALARIO > 1000;

- ¿Qué productos del almacén tienen un precio mayor que 1.000?

  SELECT * FROM ALMACEN WHERE PRECIO > 1000;

Para obtener todos estos datos, será necesario hacer uso de diferentes operadores combinados en las consultas SQL.

/ 15. Bibliografía

Connolly, T., y Begg, C. (2005). *Sistemas de Bases de Datos (5ª ed.)*. Madrid, España. Addison Wesley.

Elmasri, R., y Navathe, S. (2007). *Fundamentos de sistemas de bases de datos*. Addison Wesley.

López, I.; Castellano, M. J., y Ospino, J. (2011). *Bases de datos.* Garceta.

Oppel, A. (2009). *Databases A Beginner's Guide*. McGraw-Hill Education.

Pérez Marqués, M. (2016). *Administración básica de bases de datos con Oracle 12c SQL*. Madrid, España: Alfaomega.

Sánchez, G. C. (2001). *Sistemas gestores de bases de datos.* Paraninfo.